

Media Enrichment Extension (NodeTool-First Architecture)

Step-by-Step Technical Implementation & Architectural Delta Playbook (v5.0)

1. Codebase Baseline & Governing System Rules

This playbook outlines the exact engineering steps to migrate the existing codebase into the client's v5.0 NodeTool-First architecture. **No development time is allocated for features already built.** Below is the baseline status and system rules:

- **Scrapers (100% COMPLETE - 0 Days):** All 28 brand scrapers in scrapers/router.py are operational. No scraper rewrites needed.
- **Image Standardization Base (100% FUNCTIONAL):** Background removal (Pixelbin/fal) and canvas framing (1000x880 transparent hero, 1760x1100 white detail) already built. *Delta: Plug into n8n intake chain + save to S3.*
- **MCP Base (100% FUNCTIONAL):** Native Python MCP server (mcp_server.py) operational. *Delta: Add Jobs table tool + configure Tri-MCP with NodeTool and FableCut.*
- **Recommendation Engine (100% READY):** 3-tier vector matcher + FBT bundle engine in recommendation_engine.py. *Delta: Live sync to Shopify metafields.*
- **Strict Invariant:** Owned storage first (S3/MinIO asset_v{n}) → Baserow update. Never save temporary provider URLs.
- **The 'Edited' Lock:** Any human/agent edit in NodeTool/FableCut sets enrichment_status = 'edited', permanently protecting it from automation overwrites.

2. Phase 0: Standalone NodeTool Playground Sandbox Validation (2.5 Working Days)

Objective: Validate NodeTool in a standalone Docker sandbox before touching production Baserow or n8n (\$6).

Step 0.1: Standalone Docker Deployment

Deploy the official NodeTool container on the server/local test machine behind local reverse proxy:

```
docker run -d --name nodetool-playground -p 3000:3000 \
-e FAL_KEY="your_fal_api_key" \
-e STORAGE_LOCAL_PATH="/data/nodetool" \
-v nodetool_data:/data/nodetool \
ghcr.io/nodetool-ai/nodetool:latest
```

Step 0.2: Execution of 6 Mandatory Test Gates (\$6)

Load 10 real furniture products (Spectrum Design, Leolux, Artifort) and execute the 6 validation criteria:

- **Gate 1 (Canvas Image Workflows):** Construct visual node graph for lifestyle scene composition using FLUX / fal.ai. Measure generation speed and visual realism.
- **Gate 2 (Sketch Editor Masking - Highest Risk):** Open real furniture image in Layered Sketch Editor. Draw precision mask around furniture and execute inpainting background swap. Ensure furniture shape and upholstery remain 100% unaltered.
- **Gate 3 (Video Timeline for Humans):** Import generated footage into multi-track timeline. Perform clip trims, cuts, multi-layer arrangement, and verify clean video export.
- **Gate 4 (Timeline-Prompted Ad Generation):** Author timestamped shot list in NodeTool using Seedance 2.0 with @product reference image anchors. Verify product fidelity across shots.
- **Gate 5 (Headless Execution via API/CLI):** Execute lifestyle composition workflow via curl / CLI with dynamic JSON inputs. Confirm output image URLs are retrievable programmatically.
- **Gate 6 (NodeTool MCP Connectivity):** Connect NodeTool MCP to Claude Cowork. Verify Claude can inspect workflows and trigger headless generation conversationally.

Step 0.3: Fallback Activation Protocols (§7)

If any gate fails during Phase 0, immediately activate the pre-decided component fallback:

- *Gate 2 Fails (Masking lag/inaccuracy)* → Deploy **InvokeAI** on an on-demand Vast.ai GPU (~\$0.25/hr booted by n8n on session start).
- *Gate 3 Fails (Timeline unstable)* → Use **FableCut** as the human video editor too (via active project JSON swap wrapper).
- *Gate 5 Fails (Headless execution flaky)* → Move creative logic into **n8n direct fal.ai API calls** with Baserow prompt templates.

3. Phase 1: Storage Setup, Baserow Schema Expansion & n8n System Glue (2 Working Days)

Objective: Establish owned S3 storage, create Jobs & Template tables in Baserow, and configure n8n triggers.

Step 1.1: Provision S3 / MinIO Asset Storage

1. Create dedicated bucket binnen-catalog-assets with public read / CDN policy.

2. Configure standardized versioning path conventions:

/products/{row_id}/hero/asset__v{n}.png

/products/{row_id}/lifestyle/variant_{k}__v{n}.jpg

/products/{row_id}/video/showcase__v{n}.mp4

/products/{row_id}/3d/model__v{n}.glb & model__v{n}.usdz

Step 1.2: Baserow Schema Expansions

1. **Modify Products Table (Table 742):** Add URL fields (clean_hero_url, lifestyle_variants, lifestyle_chosen_url, video_showcase_url, model_3d_glb_url, model_3d_usdz_url), status fields (enrichment_status:

pending/processing/completed/edited/error, edited_at), trigger field (operation_marker: run_lifestyle/run_video/run_3d), and Seam formula fields (seam_image_url, seam_video_url, seam_3d_url).

2. **Create 'Jobs' Table:** id (Auto), operation (Single select), status (queued/running/done/failed), target_row_ids (Long text/JSON), template_id (Link to template tables), parameters (Long text JSON), progress_counter (Number), total_items (Number), created_at, completed_at.

3. **Create 'Prompt_Scene_Templates' Table:** template_name, category_scope, prompt_template_text, negative_prompt, aspect_ratio, model_tier.

4. **Create 'Video_Templates' Table:** template_name, type (Functional/Lifestyle), timeline_prompt_recipe, remotion_composition_id, overlay_schema.

Step 1.3: n8n Workflow Architecture

- **Workflow 1 (Entry Point A - Product Intake):** Webhook on new product in Baserow → Creates Jobs row for default clean_hero derivation.

- **Workflow 2 (Entry Point B - Grid Multi-Select Marker):** Webhook listening to operation_marker changes → Collects all marked product IDs → Creates unified Jobs row → Resets operation_marker to empty.

- **Workflow 3 (Unified Job Queue Runner):** Polls/triggers on new Jobs row → Iterates through product IDs with rate-limit awareness → Executes NodeTool headless API / fal.ai → Uploads result to S3 → Writes owned URL to Baserow → Increments progress_counter.

4. Phase 2: The Editor Seam Service & Rewired Image Pipelines (2.5 Working Days)

Objective: Deploy The Editor Seam and rewire existing image resizing logic into the automated intake & correction loop.

Step 2.1: Build The Editor Seam Microservice (FastAPI)

The Editor Seam is the thin custom service connecting Baserow, human editing surfaces, and S3 storage:

1. **Open Endpoint (GET /open?type={type}&id={row_id}):** Reads Baserow product row → Loads current asset into NodeTool board / FableCut project / <model-viewer> page.

2. **Watched-Folder / Push Endpoint (POST /export-upload):** Listens for exported files from editors → Generates new version filename (asset__v{n+1}) → Uploads to S3 → Executes Baserow PATCH: enrichment_status = 'edited', edited_at = now().

Step 2.2: Rewiring Existing Image Processing to Intake Chain

- **What Exists in Code:** fal_image_processor.py already handles transparent hero framing (1000x880) and white canvas detail framing (1760x1100).

- **Required Delta:** Wrap this existing logic into the automated n8n intake chain calling NodeTool Headless API. Save final image to S3 (asset__v1.png) and write S3 URL to Baserow clean_hero_url.

- **On-Demand Generative Tier:** Reads prompt recipe from Prompt_Scene_Templates → Generates 2–3 lifestyle variants via NodeTool/FLUX → Stores in lifestyle_variants for admin grid selection.

Step 2.3: 3-Tier Image Correction Loop Validation

Validate all 3 correction paths on staging:

1. **Approve:** Admin clicks approve in grid → Status set to completed → Pushed to Shopify.
2. **Reject & Re-render:** Parameter escalation (higher model tier, different template) without code changes.
3. **Edit:** Click Baserow formula link → Opens NodeTool sketch editor → Inpaint/composite → Export → Auto-saved to S3 and locked as edited.

5. Phase 3: Conversational Claude Cowork & Tri-MCP Suite (2 Working Days)

Objective: Enable seamless conversational catalog management, bulk jobs, and JSON video edits via Claude.

Step 3.1: Upgrading Existing Python MCP Server

- **What Exists in Code:** `mcp_server.py` already provides Baserow and Shopify tools.
- **Required Delta:** Add `create_enrichment_job` and `get_job_status` tools to `mcp_server.py`.
- **Tri-MCP Configuration:** Configure Claude Cowork / Desktop config (`claude_desktop_config.json`) to load:
 1. **Baserow MCP:** Catalog search & Job creation (Entry Point C).
 2. **NodeTool MCP:** Inspect and trigger creative canvas workflows.
 3. **FableCut MCP:** Inspect and patch JSON video timelines with conflict-safe revisions.

Step 3.2: Agentic Conversational Editing Workflow

Cowork User Prompt: *'Find all Leolux sofas missing lifestyle images and generate Japandi living room scenes.'* → Claude executes Baserow MCP search → Creates row in Jobs table → n8n executes in bulk → Live progress displays in Baserow grid.

6. Phase 4: Automated Video Production Pipeline (3.5 Working Days)

Objective: Build dual-class video engine (Functional vs Lifestyle Ads) with strict zero-text baking.

Step 4.1: Functional Videos (Programmatic Assembly)

1. Author Remotion / FFmpeg composition templates: Approved stills + short i2v motion clips + dynamic brand overlays.
2. **Overlay Pass:** Product title, price card, dimensions, and CTA rendered dynamically over footage. Bulk-safe and 100% translatable.

Step 4.2: Lifestyle Ads (Timeline-Prompt Generation)

1. Configure Video_Templates with structured shot recipes for Seedance 2.0 / FLUX 3:
[0-3s: Wide shot {room_style}] [3-7s: Smooth dolly zoom on {product_name} fabric] [7-10s: Ambient light pan]
2. Inject @product approved image URLs into reference slots to lock furniture appearance across shots.
3. Connect FableCut JSON timeline for conversational agent edits and NodeTool timeline for manual trims.

7. Phase 5: 3D Asset Pipeline Integration & (2.5 Working Days)

Objective: Automate 2D-to-3D generation, asset compression, and interactive review.

Step 5.1: 2D-to-3D Generation & Post-Processing

1. Connect fal image-to-3D (Hunyuan-class Rapid ~\$0.225; Pro ~\$0.375 on escalation). Multi-angle photos passed if available.
2. **n8n Post-Processing:** Execute `gltf-transform optimize input.glb output.glb` (mesh decimation and KTX2 texture compression) + `usdzconvert` for iOS AR QuickLook.
3. Upload optimized GLB and USDZ files to owned S3 storage.

Step 5.2: Standalone Review Page

Deploy lightweight HTML5 <model-viewer> interface: Admin inspects 3D mesh → Clicks **Approve** (syncs to Baserow & Shopify), **Reject & Escalate** (re-runs with Pro model), or **Download** (for Blender deep fixes).

8. Phases 6 to 9: Downstream Integrations & Production Launch

Phase 6: Recommendation Engine Live Sync (1 Working Day)

- **What Exists:** recommendation_engine.py (3-tier vector matcher + FBT bundle logic) is 100% complete.
- **Required Delta:** Run sync_recommendations_to_shopify.py to populate Shopify metafields.custom.similar_products and FBT metafields on live storefront.

Phase 7: Composite Multi-Product Bundle Engine (2.5 Working Days)

- Build canvas layout engine to arrange FBT pairs (e.g. Sofa + Coffee Table + Rug) onto a unified room background.
- Apply contact-shadow filters and edge-blending via NodeTool/fal inpainting for realistic lighting cohesion.
- Connect bundle generator into n8n automated batch workflows for Shopify collection pages.

Phase 8: Metadata, Options & Dynamic QR Codes (2 Working Days)

- Implement dynamic vector QR code generator linking directly to live Shopify PDPs with UTM campaign tracking.
- Configure backend variation logic separating configurable variants from standalone products and sync QR assets to Shopify.

Phase 9: AI Shopping Assistant Chatbot & Final Go-Live (3.5 Working Days)

- Vectorize complete product catalog (embeddings for titles, Dutch descriptions, 13 style vectors, prices, and stock).
- Embed AI Shopping Assistant chatbot widget on storefront frontend.
- Conduct full-scale end-to-end regression testing across Baserow, n8n, S3, NodeTool, and Shopify webshops. Execute live production cutover!

9. Production Deployment Checklist & Environment Matrix

Service / Component	Deployment Method	Key Environment Variables / Auth	Primary Port / Path
Baserow	Hetzner Dedicated Server	BASEROW_URL, BASEROW_TOKEN, Table IDs (742, Jobs)	Port 443 (HTTPS)
NodeTool	Docker (Self-Hosted)	FAL_KEY (BYOK), SECRET_STORE	Port 3000
The Editor Seam	FastAPI / Docker	BASEROW_TOKEN, S3_ACCESS_KEY, S3_SECRET_KEY	Port 8000 (Internal)
FableCut	Docker (Self-Hosted)	MCP_PORT, Project JSON storage volume	Port 4000
Owned Storage	AWS S3 / MinIO	Bucket: binnen-catalog-assets (Public read / CORS enabled)	S3 HTTPS endpoint
n8n Orchestrator	Docker / Hetzner	Baserow Credential, S3 Credential, NodeTool API Credential	Port 5678
Shopify Stores	Live REST / GraphQL	SHOPIFY_ADMIN_TOKEN, Store Targets: Woonbloq (1), Binnen (3)	Shopify Cloud

Next Engineering Action: Proceed with Phase 0 Task 0.1 (NodeTool Docker sandbox setup and 6-gate validation).